FPT Education

FPT POLYTECHNIC

cdio™

Conceive Design Implement Operate

Bài 8: Xây Dựng Luồng Tiền Xử Lý

www.poly.edu.vn

AGENDA

- Tổng quan pipeline
- Pipeline theo dữ liệu
- Kiểm thử pipeline
- Thực hành

1. Tổng Quan Luồng

1.1 Vấn đề thực tế

Trong quá trình tiền xử lý dữ liệu:

- Lỗi thủ công: Xử lý thủ công từng bước dễ dẫn đến sai sót
- Không tái sử dụng: Không thể áp dụng lại cho dữ liệu mới.
- Khó kiểm tra: Kiểm tra và so sánh các kết quả khó khăn

-> Luồng (Pipeline) giúp tự động hóa toàn bộ quy trình tiền xử lý, đảm bảo tính đồng nhất và giảm lỗi.

Ví dụ: Một dự án dự báo giá nhà có nhiều loại dữ liệu (số, categorical, ảnh, thời gian), thay vì xử lý từng bước thủ công, pipeline giúp tự động hóa và tái sử dụng quy trình xử lý cho các bộ dữ liệu mới.

1.2 Liên kết kiến thức

Pipeline tích hợp kiến thức từ các module trước:

- Module 1: Làm sạch dữ liệu
- Module 2: Xử lý outliers
- Module 3: Chuẩn hóa số
- Module 4: Xử lý dữ liệu categorical & text
- Module 5: Xử lý thời gian (time series)
- Module 6: Xử lý ảnh

1.2 Liên kết kiến thức

Ví dụ: Tiền xử lý bộ dữ liệu giá nhà

1. Làm sạch: Loại bỏ giá trị thiếu.
2. Xử lý outliers: Loại bỏ các giá trị ngoại lai trong giá trị nhà.
3. Chuẩn hóa: Quy đổi giá trị thành chuẩn để mô hình dễ dàng học.

1.3 Pipeline trong Machine Learning

- Là một chuỗi các bước tiền xử lý dữ liệu và huấn luyện mô hình được kết nối với nhau thành một quy trình tự động hóa.
- Mỗi bước trong pipeline có thể là một transformer (biến đổi dữ liệu) hoặc estimator (mô hình học máy).
- Mục tiêu chính của pipeline là tạo ra một quy trình tái sử dụng và chuẩn hóa cho toàn bộ quy trình học máy, từ tiền xử lý đến huấn luyện và đánh giá mô hình.

1.3 Pipeline trong Machine Learning

- Transformers (Biến đổi dữ liệu)

- Imputation: Điền giá trị thiếu (Missing Value Imputation)
- Scaling/Normalization: Chuẩn hóa giá trị dữ liệu, đưa dữ liệu về một phạm vi nhất định, như Min-Max Scaling hoặc Standardization.
- Encoding: Chuyển đổi dữ liệu categorical thành dạng số, như One-Hot Encoding, Label Encoding.
- Feature Engineering: Tạo ra các tính năng mới từ dữ liệu hiện có.
- Dimensionality Reduction: Giảm số chiều của dữ liệu, như PCA (Principal Component Analysis).
- Text Vectorization: Biến đổi văn bản thành các đặc trưng số (ví dụ, dùng TF-IDF hoặc Word2Vec).

1.3 Pipeline trong Machine Learning

- Estimators (Huấn luyện mô hình)

- Models: Đây là các mô hình học máy như hồi quy tuyến tính, cây quyết định, Random Forest, SVM, hoặc các mô hình học sâu (Deep Learning).
- Evaluators: Các công cụ đánh giá mô hình, chẳng hạn như k-fold cross-validation hoặc đánh giá với các chỉ số như accuracy, precision, recall, F1-score.

1.3 Pipeline trong Machine Learning

- Ví dụ dự báo giá nhà

Bước 1 - Tiền xử lý dữ liệu:

- Điền giá trị thiếu: Nếu có cột diện tích hoặc giá trị thiếu trong bộ dữ liệu, bạn có thể điền giá trị trung bình hoặc trung vị.
- Loại bỏ outliers: Dữ liệu giá nhà có thể chứa các giá trị ngoại lai không hợp lý (ví dụ, giá nhà quá thấp hoặc quá cao so với mặt bằng chung), pipeline sẽ tự động loại bỏ chúng.
- Chuẩn hóa: Các cột như diện tích có thể có đơn vị lớn (ví dụ: hàng ngàn mét vuông), cần phải chuẩn hóa để đưa vào mô hình học máy.

1.3 Pipeline trong Machine Learning

- Ví dụ dự báo giá nhà

Bước 2 - Xử lý dữ liệu categorical: Cột vị trí có thể có giá trị như "Hà Nội", "TPHCM", pipeline sẽ thực hiện One-Hot Encoding để biến chúng thành các đặc trưng nhị phân (0 hoặc 1).

Bước 3 - Chuyển đổi và huấn luyện mô hình: Đưa dữ liệu vào mô hình học máy Linear Regression để huấn luyện.

Bước 4 - Đánh giá mô hình: Mô hình sẽ được kiểm tra và đánh giá bằng các chỉ số như MSE hoặc R^2.

2. Pipeline Theo Dữ Liệu

2.1 Dữ liệu số (Numerical)

- Dữ liệu số thường có giá trị khác nhau về phạm vi (ví dụ: tuổi 0–100, thu nhập 0–1 triệu).
- Nếu đưa trực tiếp vào mô hình (như Linear Regression, Neural Network), các giá trị lớn sẽ chi phối mô hình, dẫn đến hiệu quả kém.
- Dữ liệu số thường có giá trị thiếu hoặc ngoại lai, nếu không xử lý sẽ làm mô hình bị sai lệch.

2.1 Dữ liệu số (Numerical)

- Các bước xử lý:

1. Xử lý giá trị thiếu: điền bằng trung bình, trung vị, hoặc dự đoán.
2. Chuẩn hóa / Scaling:

- StandardScaler: (x - mean) / std
- MinMaxScaler: (x - min) / (max - min)

1. Xử lý ngoại lai: loại bỏ hoặc cắt giá trị ngoài ngưỡng
2. Biến đổi nâng cao: log, sqrt, polynomial features

2.2 Dữ liệu phân loại (Categorical)

- Máy tính không hiểu "Nam" hay "Nữ", cần chuyển thành số.
- Một số biến có thứ tự (thấp, trung bình, cao) cần giữ thông tin thứ tự, còn biến nominal thì không.
- Một số giá trị hiếm nếu không xử lý có thể gây overfitting hoặc làm mô hình không tổng quát.

2.2 Dữ liệu phân loại (Categorical)

- Các bước xử lý:

1. Xử lý giá trị thiếu: điền giá trị phổ biến, hoặc “unknown”
2. Mã hóa (Encoding):

- One-hot encoding cho dữ liệu nominal (không thứ tự)
- Label/Ordinal encoding cho dữ liệu có thứ tự

1. Xử lý dữ liệu hiếm: nhóm các giá trị ít gặp thành “Other”

2.3 Dữ liệu văn bản (Textual)

- Máy tính không hiểu trực tiếp chữ viết, cần chuyển đổi văn bản thành vector số.

- Dữ liệu thô có nhiễu: dấu câu, số, ký tự đặc biệt, từ phổ biến không mang ý nghĩa.

2.3 Dữ liệu văn bản (Textual)

- Các bước xử lý:

1. Làm sạch văn bản: loại bỏ dấu câu, số, ký tự đặc biệt
2. Tokenization: tách câu → từ
3. Loại bỏ stopwords: từ không mang ý nghĩa
4. Vector hóa: Bag-of-Words, TF-IDF, word embeddings

2.4 Dữ liệu chuỗi thời gian (Time Series)

- Dữ liệu thời gian có trật tự, có thể có trend, seasonality.

- Không thể xử lý như dữ liệu độc lập; cần thông tin lịch sử và đặc trưng thời gian.

2.4 Dữ liệu chuỗi thời gian (Time Series)

- Các bước xử lý:

1. Xử lý giá trị thiếu: điền bằng phương pháp nội suy, forward-fill
2. Chuyển đổi đặc trưng thời gian:

- Ngày → Tháng, Thứ, Quý
- Lấy mùa vụ, trend

1. Scaling / Normalization: chuẩn hóa dữ liệu theo range hoặc Z-score
2. Feature Engineering nâng cao: Rolling mean, lag features, differences

2.5 Dữ liệu hình ảnh (Image)

- Mỗi hình ảnh là một ma trận pixel; giá trị pixel đa dạng.
- Mô hình học sâu yêu cầu kích thước đồng nhất và giá trị chuẩn hóa để học hiệu quả.
- Augmentation giúp mô hình khả năng tổng quát với các biến thể của hình ảnh.

2.5 Dữ liệu hình ảnh (Image)

- Các bước xử lý:

1. Resize / Crop: đồng nhất kích thước
2. Chuẩn hóa pixel: chia 255, chuẩn hóa theo mean/std
3. Augmentation (tùy chọn): xoay, lật, thay đổi độ sáng, zoom
4. Chuyển đổi sang tensor: sẵn sàng cho mô hình deep learning

3. Đánh Giá & Hoàn Thiện

3. ĐÁNH GIÁ & HOÀN THIỆN

- Đảm bảo các bước tiền xử lý chạy đúng thứ tự và không sinh lỗi.
- Dữ liệu đầu ra sau pipeline đúng định dạng, đúng số chiều, phù hợp với mô hình.
- Phát hiện sớm lỗi ở từng bước (missing, encoding, scaling, outliers).
- Giảm rủi ro khi đưa pipeline sang dữ liệu thực tế hoặc triển khai production.

3.1 Kiểm thử phân phối dữ liệu (Data Distribution Checking)

- Mục tiêu: đảm bảo pipeline không làm méo mó dữ liệu một cách bất thường. Phát hiện các biến đổi bất thường: chuẩn hóa sai, encoding lỗi, mất thông tin,...
- Nội dung kiểm tra:

1. So sánh thống kê mô tả trước/sau tiền xử lý:

- Mean, median, std, min–max
- Tỷ lệ giá trị thiếu, số giá trị outlier

3.1 Kiểm thử phân phối dữ liệu (Data Distribution Checking)

2. Quan sát trực quan:

- Histogram để kiểm tra thay đổi phân phối
- Boxplot để kiểm tra outlier và độ lan truyền
- Density plot cho dữ liệu liên tục

2. Kiểm định thống kê (nếu cần):

- KS-test, Shapiro-Wilk để so sánh phân phối
- Chi-square test cho categorical

3.2 Chạy thử pipeline trên tập dữ liệu nhỏ (Smoke Test)

- Mục tiêu: đảm bảo pipeline chạy được, không lỗi, không làm sai hình dạng dữ liệu. Phát hiện lỗi sớm trước khi chạy full dataset.
- Nội dung kiểm tra:

- Hình dạng (shape) trước/sau: số dòng, số cột, tên cột
- Kiểu dữ liệu (data types): float/int/object sau encode
- Null/NaN bất thường: kiểm tra imputer có hoạt động đúng
- Giá trị output tạm thời: kiểm tra scaler, encoder, tính toán feature
- Thời gian chạy: phát hiện bước quá nặng hoặc vòng lặp vô hạn

3.3 Cross-validation kết hợp pipeline (End-to-end Evaluation)

- Mục tiêu: đánh giá khả năng tổng quát hoá của toàn bộ quy trình tiền xử lý + model.
- Nội dung kiểm tra:

- Gói toàn bộ quy trình vào Pipeline / ColumnTransformer
- Chạy k-fold cross-validation để đo hiệu suất ổn định
- Tránh data leakage vì pipeline đảm bảo các bước (impute/scale/encode) chỉ fit trên training fold
- So sánh hiệu suất của nhiều pipeline khác nhau

3.4 Kiểm thử từng bước độc lập (Component-wise Testing)

- Mục tiêu: debug chi tiết từng thành phần mà pipeline sử dụng, đảm bảo mỗi block xử lý hoạt động đúng kỳ vọng.
- Nội dung kiểm tra:

1. Imputer (trám giá trị trống)

- Kiểm tra logic: mean/median/constant phù hợp loại dữ liệu
- Kiểm tra không tạo ra giá trị NaN sau khi fit-transform
- So sánh giá trị trám với thống kê gốc

3.4 Kiểm thử từng bước độc lập (Component-wise Testing)

2. Encoder (OneHot, LabelEncoder, TargetEncoder)

- So sánh số lượng category trước/sau
- Kiểm tra xuất hiện category chưa từng gặp (unknown handling)
- Đảm bảo không lệ thuộc thứ tự category nếu không mong muốn

2. Scaler/Normalizer

- Kiểm tra mean=0/std=1 (StandardScaler)
- Kiểm tra min=0/max=1 (MinMaxScaler)
- So sánh phân phối trước–sau để tránh scaling sai

3.4 Kiểm thử từng bước độc lập (Component-wise Testing)

4. Outlier Logic

- Phát hiện xem outlier có bị loại/clip đúng kỳ vọng
- Kiểm tra tỉ lệ dữ liệu bị loại bỏ
- Đảm bảo không ảnh hưởng quá lớn đến phân phối

4. Feature Engineering

- Kiểm tra formula, phép toán, unit
- Kiểm tra feature mới tạo ra không toàn NaN
- So sánh correlation trước–sau để đánh giá hữu ích

3.5 Kiểm thử tính ổn định

- Kiểm tra pipeline chạy lặp lại cho kết quả giống nhau
- Set random_state ở các bước liên quan
- So sánh kết quả giữa nhiều batch dữ liệu khác nhau

3.6 Kiểm thử với dữ liệu bất thường (Edge-case Testing)

- Dữ liệu có missing 100% 1 cột
- Dữ liệu có category chưa từng xuất hiện
- Dữ liệu số toàn bằng 0
- Dữ liệu noise cao
- Các trường không xuất hiện trong training

Tổng hợp

- Kiểm thử phân phối: thống kê + biểu đồ
- Smoke test dataset nhỏ: kiểm tra shape/dtypes/null
- Cross-validation end-to-end: đánh giá tổng thể pipeline
- Kiểm thử từng bước: imputer, encoder, scaler, outlier,…
- Kiểm thử tái tập: kết quả ổn định, có random_state
- Edge-case testing: dữ liệu lỗi, thiếu, unseen categories

4. Thực Hành

Chuẩn bị thư viện

```
import pandas as pd
import numpy as np

from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer

from sklearn.preprocessing import OneHotEncoder, StandardScaler, FunctionTransformer
from sklearn.feature_extraction.text import TfidfVectorizer

from sklearn.impute import SimpleImputer
from sklearn.ensemble import RandomForestRegressor

from sklearn.model_selection import train_test_split, cross_val_score
```

Chuẩn bị dữ liệu

```python
data = {
    "area": [50, 70, 65, np.nan, 120],
    "price": [2e9, 3e9, np.nan, 1.5e9, 6e9],
    "rooms": [2, 3, 2, 1, 4],
    "location": ["A", "B", "A", "C", "B"],
    "house_type": ["Townhouse", "Villa", "Apartment", "Apartment", None],
    "description": [
        "Nhà đẹp gần trung tâm",
        "Biệt thự cao cấp, có sân vườn",
        "Căn hộ giá tốt cho gia đình trẻ",
        "Nhà nhỏ phù hợp vợ chồng mới cưới",
        "Biệt thự rộng rãi, tiện nghi"
    ],
    "date": ["2021-01-15", "2021-02-20", "2021-03-10", "2021-02-01", "2021-04-03"]
}

df = pd.DataFrame(data)
df["date"] = pd.to_datetime(df["date"])

df
```

Chuẩn bị dữ liệu

```python
data = {
    "area": [50, 70, 65, np.nan, 120],
    "price": [2e9, 3e9, np.nan, 1.5e9, 6e9],
    "rooms": [2, 3, 2, 1, 4],
    "location": ["A", "B", "A", "C", "B"],
    "house_type": ["Townhouse", "Villa", "Apartment", "Apartment", None],
    "description": [
        "Nhà đẹp gần trung tâm",
        "Biệt thự cao cấp, có sân vườn",
        "Căn hộ giá tốt cho gia đình trẻ",
        "Nhà nhỏ phù hợp vợ chồng mới cưới",
        "Biệt thự rộng rãi, tiện nghi"
    ],
    "date": ["2021-01-15", "2021-02-20", "2021-03-10", "2021-02-01", "2021-04-03"]
}

df = pd.DataFrame(data)
df["date"] = pd.to_datetime(df["date"])

df
```

Phân biệt dữ liệu

```
num_cols = ["area", "rooms"]
cat_cols = ["location", "house_type"]
text_col = "description"
date_col = "date"  # sẽ được xử lý bằng time_series
target = "price"
```

Phân biệt dữ liệu

```
num_cols = ["area", "rooms"]
cat_cols = ["location", "house_type"]
text_col = "description"
date_col = "date"  # sẽ được xử lý bằng time_series
target = "price"
```

Chuẩn bị pipeline

```
num_pipeline = Pipeline([
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", StandardScaler()),
])
cat_pipeline = Pipeline([
    ("imputer", SimpleImputer(strategy="most_frequent")),
    ("onehot", OneHotEncoder(handle_unknown="ignore"))
])
text_pipeline = Pipeline([
    ("tfidf", TfidfVectorizer())
])
```

Chuẩn bị pipeline

```
def extract_time_features(df):
    df = df.copy()
    df["year"] = df["date"].dt.year
    df["month"] = df["date"].dt.month
    return df[["year", "month"]]

time_series_transformer = FunctionTransformer(extract_time_features, validate=False)

time_pipeline = Pipeline([
    ("extract", time_series_transformer),
    ("scaler", StandardScaler())
])
```

Sử dụng Column Transformer

```
full_preprocess = ColumnTransformer([
    ("num", num_pipeline, num_cols),
    ("cat", cat_pipeline, cat_cols),
    ("text", text_pipeline, text_col),
    ("time", time_pipeline, [date_col])
])
```

Áp dụng mô hình

```
model = Pipeline([
    ("preprocess", full_preprocess),
    ("regressor", RandomForestRegressor(n_estimators=100, random_state=42))
])
df_clean = df.dropna(subset=[target])
df_clean = df_clean.replace({None: np.nan})
X = df_clean.drop(columns=[target])
y = df_clean[target]
model.fit(X, y)
print("Train score:", model.score(X, y))
```

Thử nghiệm dữ liệu mới

```
new_data = pd.DataFrame({
    "area": [80],
    "rooms": [3],
    "location": ["A"],
    "house_type": ["Villa"],
    "description": ["Nhà đẹp gần trường học, phù hợp gia đình"],
    "date": [pd.to_datetime("2021-05-01")]
})

pred = model.predict(new_data)
pred
```

FPT Education

FPT POLYTECHNIC

THANK YOU